

3.IMAGE ENHANCEMENT USING INTENSITY TRANSFORMATIONS

AIM:

To study and perform image enhancement using the basic gray-level point transformations.

INTRODUCTION:

The principal objective of image enhancement is to process an image so that the result is more suitable than the original image for a specific application. The word ‘**specific**’ implies that the techniques discussed below are problem-oriented.

Image enhancement can be done in two domains:

1. **Spatial Domain**
2. **Transform Domain**

In **spatial domain image processing**, there are two main approaches:

- **Point processing** – Single pixel is processed at a time
- **Neighborhood processing (Mask processing)** – Convolution of a 3×3, 5×5, or other-sized mask with the image

POINT OPERATIONS:

Point operations are also referred to as **gray-level transformations** or **spatial transformations**. These are among the simplest enhancement techniques. They can be expressed as:

$$G(x,y)=T(F(x,y))$$

Where:

- $F(x,y)$ is the original image
- $G(x,y)$ is the transformed image

It can also be expressed as:

$$s=T(r)$$

Where:

- T is the transformation function
- r is the original pixel value
- s is the transformed pixel value

The transformation function T maps pixel value r into pixel value s. These functions are typically implemented using **look-up tables** (LUTs). For an 8-bit grayscale image, the LUT holds values between **0 and 255**.

LINEAR OPERATIONS:

Intensity transformations are among the simplest of all image processing techniques. As noted in the previous section, we denote the values of pixels, before and after processing, by r and s, respectively. These values are related by a transformation T, as given in Eq. (3-2), that maps a pixel value r into a pixel value s. Because we deal with digital quantities, values of an intensity transformation function typically are stored in a table, and the mappings from r to s are implemented via table lookups. For an 8-bit image, a lookup table containing the values of T will have 256 entries. As an introduction to intensity transformations, consider Fig. 3.3, which shows three basic types of functions used frequently in image processing: linear (negative and identity transformations), logarithmic (log and inverse-log transformations), and power-law (nth power and nth root transformations). The identity function is the trivial case in which the input and output intensities are identical.

NEGATIVE TRANSFORMATION:

Negative Transformation: The negative point transformation function (also known as contrast reverse). The negative transformation is used to make the output more suitable for the task at hand (e.g., by making it easier to notice interesting details in the image). The mathematical form for negative point transformation with gray levels in the range [0, L-1] is

$$s = (L-1) - r$$

Code:

```
r=imread("cameraman.tif");  
s=255-r;  
imshow(r)  
figure ,imshow(s)
```

Output:



FIGURE 1 (a) A photograph of a cameraman. (b) Negative image obtained using negative transformation.

LOGARITHMIC TRANSFORMATIONS:

The log transformation maps a narrow range of low-level grey scale intensities into a wider range of output values and similarly maps the wide range of high-level grey scale intensities into a narrow range of high level output values. The log transformation curve is given by the expression,

$$s = c \log (1 + r)$$

Where c is a constant. We would use a transformation of this type to expand the values of dark pixels in an image while compressing the higher-level values. The opposite is true of the inverse log transformation.

Code:

```
i1=imread("C:\Users\DELL\Pictures\DIP3E_Original_Images_CH03\spark.tif");  
i1=im2double(i1);  
c=15;  
res=c*log(1+i1);  
imshow(i1)  
figure,imshow(res)
```

Output:

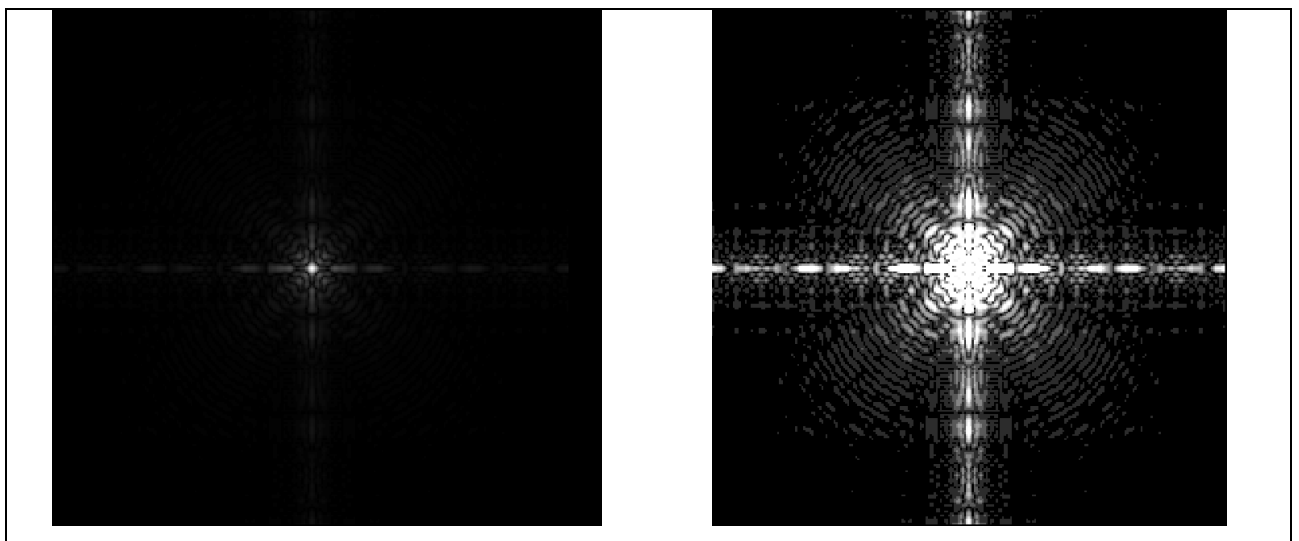


FIGURE 2 (a) Fourier spectrum displayed as a grayscale image. (b) Result of applying the log transformation with $c = 15$. Both images are scaled to the range $[0, 255]$.

POWER LAW (GAMMA) TRANSFORMATION:

The power law transformation function is described by

$$s = c r^\gamma$$

where r is the original pixel value, s is the resulting pixel value, c is a scaling constant, and γ is a positive value.

As in the case of the log transformation, power-law curves with fractional values of γ map a narrow range of dark input values into a wider range of output values, with the opposite being true for higher values of input levels. Unlike the log function, however, we notice here a family of possible transformation curves obtained simply by varying γ . By convention, the exponent in the power-law equation is referred to as gamma. The process used to correct this power-law response phenomenon is called gamma correction.

Code:

```
i1=imread("C:\Users\DELL\Pictures\DIP3E_Original_Images_CH03\spark.tif");  
i1=im2double(i1);  
c=1;  
res=c*i1.^0.5  
imshow(i1)  
figure,imshow(res)
```

Output:

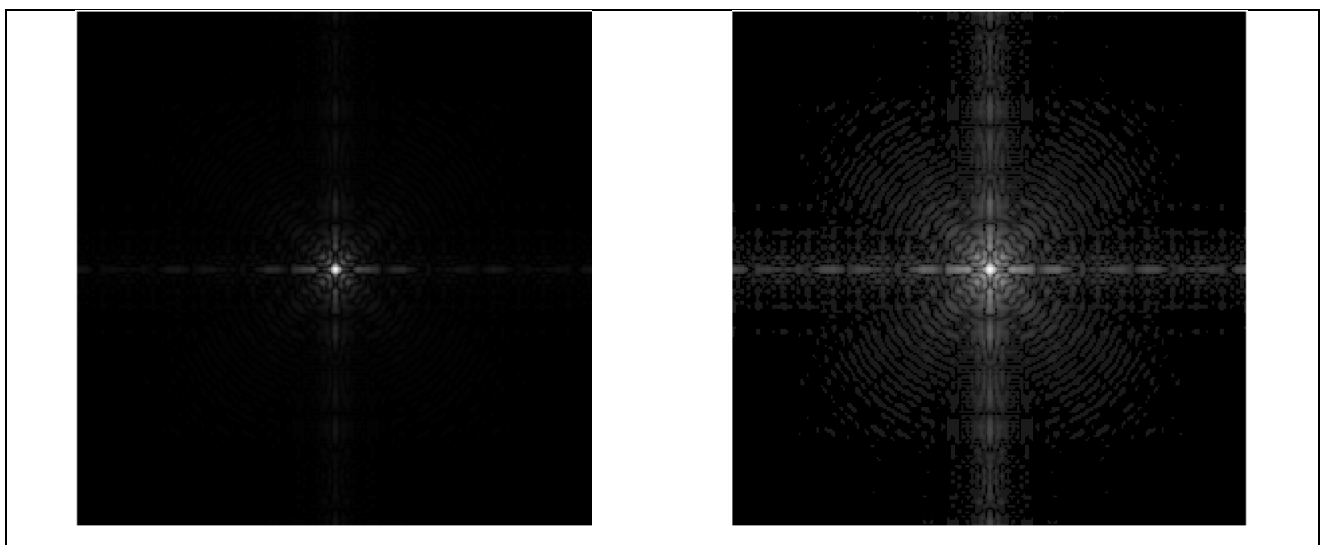


FIGURE 3 (a) Fourier spectrum displayed as a grayscale image. (b) Result of applying the gamma transformation with $c = 1$ and $\gamma = 0.5$. Both images are scaled to the range $[0, 255]$.

PIECEWISE LINEAR TRANSFORMATION FUNCTIONS:

Piecewise linear transformation functions offer a flexible and practical approach to image enhancement. Unlike fixed mathematical formulas, these functions allow us to define custom transformation curves that better suit the needs of the image being processed. Their primary application includes **contrast stretching**, **intensity-level slicing**, and **bit-plane slicing**.

1. CONTRAST STRETCHING:

Contrast stretching is a technique used to improve the visibility of details in low-contrast images, often caused by poor lighting or limited sensor dynamic range. By adjusting intensity values using a piecewise linear function, we can spread the pixel intensities across the full dynamic range (e.g., 0 to 255), enhancing overall contrast. This is done by mapping the minimum and maximum intensity levels in the image to the full displayable range.

Code:

```
clear all, close all, clc
i = imread("pout.tif");
r1 = input('Enter a value: ');
s1 = input('Enter a value: ');
r2 = input('Enter a value: ');
s2 = input('Enter a value: ');
r3 = input('Enter a value: ');
s3 = input('Enter a value: ');
for row = 1:size(i,1)
    for col = 1:size(i,2)
        y = i(row,col);
        if y <= r1
            x(row,col) = (s1/r1)*y;
        elseif y > r1 && y <= r2
```

```

        x(row,col) = ((s2 - s1)/(r2 - r1))*(y - r1) +
        s1;
    elseif y > r2 && y <= r3
        x(row,col) = ((s3 - s2)/(r3 - r2))*(y - r2) +
        s2;
    else
        x(row,col) = ((255 - s3)/(255 - r3))*(y - r3) +
        s3;
    end
end
end
x = uint8(x);
figure;
imshow(uint8(i));
figure, imshow(x);

```

Output:



FIGURE 4 Contrast stretching. (a) Piecewise linear transformation function. (b) A low-contrast electron microscope image of pollen, magnified 700 times. (c) Result of contrast stretching.

2. INTENSITY-LEVEL SLICING:

This method enhances specific intensity ranges in an image. It's particularly useful in applications like satellite imaging and X-ray flaw detection. One way is to assign bright or dark values to a selected range and suppress the rest (binary slicing). Alternatively, we can brighten or highlight only the desired range, preserving the rest of the image's intensity.

Code:

```
r=imread("C:\Users\DELL\Pictures\DIP3E_Original_Images_CH02\kidney.tif");
[ro,co]=size(r);
A=160
B=230
for i=1:ro
    for j=1:co
        if r(i,j)<A || r(i,j)>B
            g(i,j)=10;
        else
            g(i,j)=255;
        end
    end
end
imshow(r)
figure, imshow(uint8(g))
```

Output:

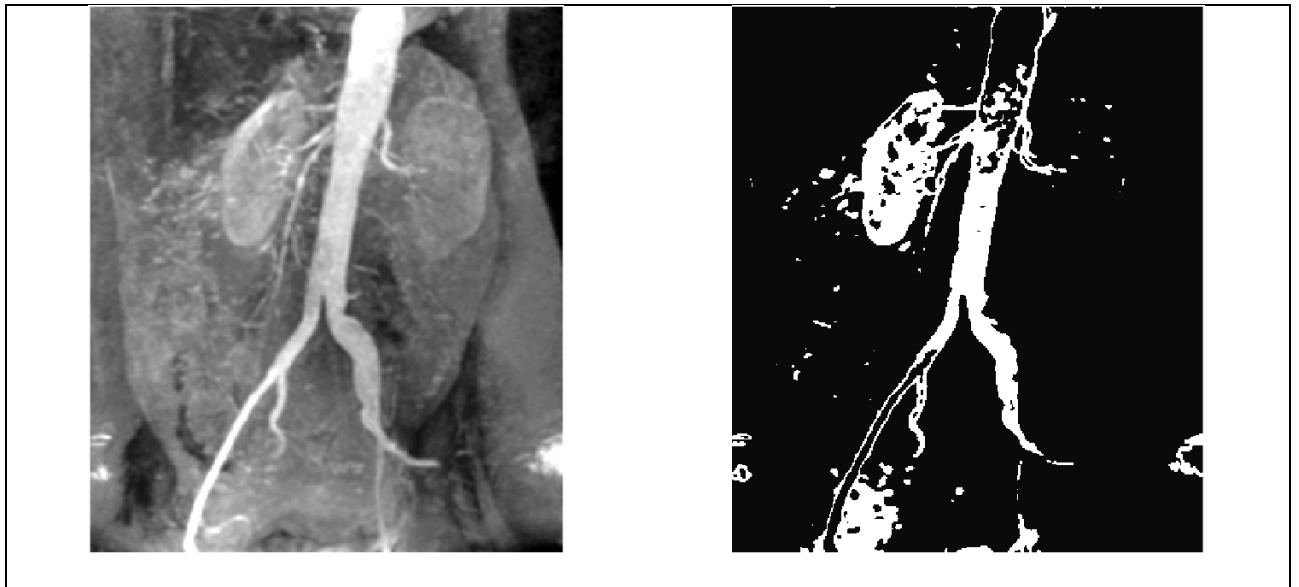


FIGURE 5 (a) Aortic angiogram. (b) Result of using a slicing transformation with the range of intensities of interest selected in the upper end of the gray scale. (Original image courtesy of Dr. Thomas R. Gest, University of Michigan Medical School.)

3. BIT-PLANE SLICING:

An image can be broken down into its binary bit components, or bit planes. Each bit plane contributes differently to the image's overall appearance. Higher-order bit planes (e.g., the most significant bits) typically contain major image features, while lower-order planes hold finer details. Bit-plane slicing helps in image analysis and data compression techniques.

Code:

```
clear all, close all, clc
i = imread("cameraman.tif")
imshow(i)
e = bitand(i, 2^1)
figure, imshow(logical(e))
f = bitand(i, 2^2)
figure, imshow(logical(f))
g = bitand(i, 2^3)
figure, imshow(logical(g))
```

```
a = bitand(i,2^4)
figure,imshow(logical(a))
b = bitand(i,2^5)
figure,imshow(logical(b))
c = bitand(i,2^6)
figure,imshow(logical(c))
d = bitand(i,2^7)
figure,imshow(logical(d))
```

Output:

